Kinektro — Conexión entre PCs (Oculus ↔ Kinect)

Especificación del protocolo UDP bidireccional entre la PC del Oculus y la PC del Kinect

Resumen

Este documento describe el canal de comunicación implementado entre las dos PCs de la experiencia Kinektro: la PC A, que corre el Oculus y el motor de audio (FMOD/DJ_Manager), y la PC B, que tiene el sensor Kinect, su propia pantalla y reconocimiento de voz para elegir efectos. El canal es UDP, bidireccional, y es independiente del canal UDP local que ya existía entre el middleware del Kinect y Unity dentro de la PC B. Se detalla la arquitectura, el protocolo de mensajes, la estrategia de confiabilidad sobre UDP, los componentes de software implementados y la configuración de red necesaria.

1. Contexto y objetivo

En el diseño original de Kinektro, el Oculus y el Kinect eran periféricos de un mismo proceso de Unity corriendo en una sola PC; el UDP local existente (puerto 5005) es solo un workaround para la incompatibilidad entre el SDK del Kinect (Microsoft.Kinect.dll) y Unity, no una comunicación real entre dos máquinas.

Para reducir la carga cognitiva de una sola persona operando la consola completa, el proyecto evolucionó a una experiencia colaborativa de dos jugadores en dos PCs distintas: el Jugador A usa el Oculus y controla la consola (selección de canciones, crossfader, etc.), mientras que el Jugador B, frente a su propia pantalla, usa el Kinect para modular la intensidad de los efectos con el gesto de levantar las manos, y elige qué efecto armar en cada canal mediante comando de voz. Esto obliga a una comunicación real entre dos máquinas en la misma red local, ya que el audio (FMOD) vive únicamente en la PC A.

El objetivo de esta pieza es, entonces: que los gestos y la selección de efectos capturados en la PC B lleguen a la PC A para alimentar el motor de audio, y que la PC A pueda avisarle a la PC B en qué momentos hay una ventana de selección de efectos abierta y cuáles están disponibles.

2. Arquitectura general

La figura 1 resume los dos procesos de Unity (uno por PC) y los componentes involucrados en cada uno. El bloque morado es el canal nuevo implementado para esta tarea.

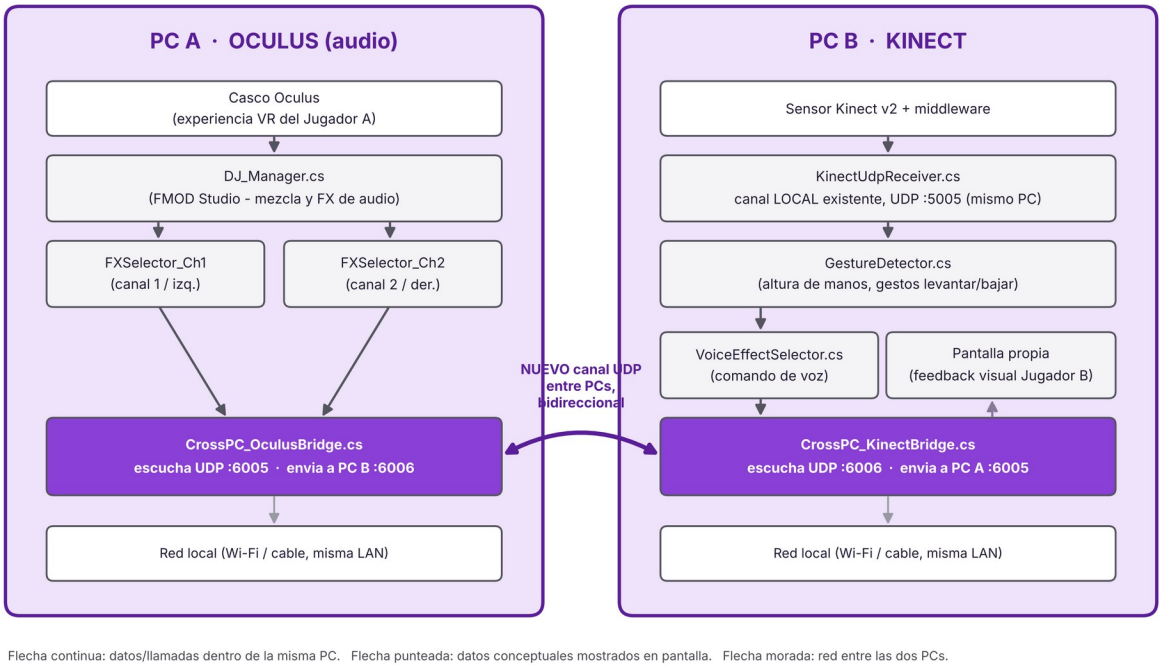


Figura 1. Arquitectura de dos PCs y ubicación del canal UDP nuevo.

3. Dos canales UDP distintos (no deben confundirse)

El proyecto usa dos canales UDP con propósitos y alcances completamente distintos. Es importante no mezclarlos en el informe ni en el código: uno es interno a la PC B y ya existía; el otro es nuevo, cruza la red local y es el que se documenta aquí.

Canal	Alcance	Dirección	Puerto(s)	Propósito
Kinect middleware → Unity (KinectUdpReceiver.cs)	Local, dentro de la PC B únicamente	Unidireccional	5005	Workaround histórico para la incompatibilidad entre Microsoft.Kinect.dll y Unity. Ya existía antes de este trabajo y no se modificó.
Puente entre PCs (CrossPC_OculusBridge.cs / CrossPC_KinectBridge.cs)	Red local (LAN), entre PC A y PC B	Bidireccional	6005 (PC A escucha) 6006 (PC B escucha)	Canal nuevo: lleva gestos, altura de mano y selección de efecto de PC B a PC A, y ventanas de efectos disponibles de PC A a PC B. Es el objeto de esta especificación.

4. Protocolo de mensajes

Los mensajes son texto plano en UTF-8, un mensaje por datagrama UDP, con los campos separados por el carácter "|". Este formato replica, por consistencia, el estilo ya usado en KinectUdpReceiver.cs (que separa con ";" y ","), y permite depurar el tráfico con herramientas simples como Wireshark o netcat sin decodificar binario.

4.1 Mensajes de PC B (Kinect) hacia PC A (Oculus)

Mensaje	Significado	Tipo de envío	Ejemplo
HAND_RAISED|LADO	La mano indicada (LEFT/RIGHT) se levantó por encima de la cabeza. Dispara ActivateSelected() del FXSelector de ese canal en PC A.	Discreto, x3 (redundante)	HAND_RAISED|LEFT
HAND_LOWERED|LADO	La mano indicada bajó. Dispara Deactivate() del FXSelector de ese canal en PC A.	Discreto, x3 (redundante)	HAND_LOWERED|RIGHT
HAND_HEIGHT|LADO|valor	Altura normalizada (0.0–1.0) de esa mano, enviada cada frame. Alimenta DJ_Manager.SetFXIntensity() en PC A.	Continuo, sin redundancia	HAND_HEIGHT|LEFT|0.73
SELECT_EFFECT|canal|slot	Efecto elegido por comando de voz en PC B para el canal indicado (1=izq./Ch1, 2=der./Ch2). Dispara FXSelector.SelectEffect(slot) en PC A.	Discreto, x3 (redundante)	SELECT_EFFECT|1|5

4.2 Mensajes de PC A (Oculus) hacia PC B (Kinect)

Mensaje	Significado	Tipo de envío	Ejemplo
WINDOW_OPEN|canal|slots	Se abrió una ventana de selección de efectos para ese canal; "slots" es la lista de efectos disponibles para la canción actual, separados por coma.	Discreto, x3 (redundante)	WINDOW_OPEN|1|1,3,5
WINDOW_CLOSE|canal	Se cerró la ventana de selección de efectos de ese canal.	Discreto, x3 (redundante)	WINDOW_CLOSE|1
FADER|canal|valor	Valor (0.0–1.0) de un fader/knob de la consola, opcional, por si la pantalla de PC B lo quiere reflejar.	Continuo, sin redundancia	FADER|2|0.50

4.3 Mapeo de índices de efecto (slot)

El índice de efecto es el mismo que ya usan FXSelector.cs y DJ_Manager.cs en el proyecto; no se introdujo una numeración nueva:

Slot	Efecto	Parámetro FMOD de intensidad continua asociado
1	Reverb	Reverb_Intensity
2	Delay	Delay_Intensity
3	Distortion	Distortion_Intensity
4	LowPass	LowPass_Intensity
5	Flanger	Flanger_Intensity

Nota: al momento de este trabajo, solo "Reverb_Intensity" está efectivamente cableado en el proyecto FMOD (prueba de concepto en la canción "Te gusto"); los demás parámetros se agregan más adelante replicando el mismo patrón en FMOD Studio. Esto no afecta el protocolo de red, que ya soporta los 5 slots.

5. Estrategia de confiabilidad sobre UDP

UDP no garantiza entrega ni orden. En vez de implementar un esquema de confirmación/reenvío (ACK + retransmisión), que agrega complejidad y tiempo de desarrollo no disponible en un proyecto de curso, se adoptó una estrategia pragmática de dos reglas:

- Valores continuos (HAND_HEIGHT, FADER): se envían una vez por frame, sin redundancia. Si se pierde un paquete, el siguiente frame lo reemplaza de inmediato; el efecto es imperceptible.
- Mensajes discretos importantes (HAND_RAISED/LOWERED, SELECT_EFFECT, WINDOW_OPEN/CLOSE): se envían 2–3 veces seguidas (redundantSends, configurable en el Inspector). Esto es seguro porque los métodos que disparan en el receptor son idempotentes: llamar ActivateSelected() o SelectEffect(slot) dos veces seguidas con el mismo valor no produce ningún efecto colateral distinto de llamarlo una vez.

Se evaluó usar WebSockets para los mensajes discretos (dan entrega confiable sobre TCP), pero se descartó por el tiempo adicional que tomaría integrarlo junto al UDP ya existente en el proyecto, y porque la redundancia simple resuelve el caso de uso dentro de una red local confiable con pérdida de paquetes baja.

6. Componentes de software implementados

Script	PC / Carpeta	Responsabilidad
GestureDetector.cs (modificado, parche aditivo)	Ambas PCs Assets/Kinect UDP/	Ya existía; se le agregaron los eventos onLeftHandHeightChanged y onRightHandHeightChanged (UnityEvent<float>), que ahora se disparan cada frame con la altura normalizada de cada mano incluso si djManager/fxSelectorCh1/fxSelectorCh2 están sin asignar. Esto permite reutilizar la misma clase en la PC B, que no tiene motor de audio local, sin duplicar el cálculo de altura. No cambia el comportamiento en las escenas existentes (BasicScene, Escena_3).
CrossPC_OculusBridge.cs	PC A Assets/Scripts_Code/	Escucha UDP en el puerto 6005 en un hilo de fondo (mismo patrón que KinectUdpReceiver.cs) y aplica los mensajes recibidos directamente sobre FXSelector_Ch1/Ch2 y DJ_Manager locales. Expone SendWindowOpen/SendWindowClose/SendFader para notificar a la PC B.
CrossPC_KinectBridge.cs	PC B Assets/Kinect UDP/	Escucha UDP en el puerto 6006. Expone los métodos públicos (OnLeftHandRaised, OnLeftHandHeightChanged, etc.) que se cablean en el Inspector a los eventos de GestureDetector, y SelectEffectByVoice(canal, slot) para el reconocimiento de voz. Guarda el estado de "ventana abierta" y efectos disponibles reportado por la PC A.
VoiceEffectSelector.cs	PC B Assets/Scripts_Code/	Reconocimiento de voz mínimo con UnityEngine.Windows.Speech.KeywordRecognizer (nativo de Windows, sin paquetes ni internet). Mapea frases fijas en español (p. ej. "reverb canal uno") a pares (canal, slot) y llama a CrossPC_KinectBridge.SelectEffectByVoice(). Si la plataforma no soporta este motor de voz, el componente se autodesactiva y el resto del puente sigue funcionando igual.

7. Flujo de ejecución — dos ejemplos paso a paso

7.1 El Jugador B levanta y mueve la mano izquierda

- GestureDetector.cs (en PC B) detecta HandLeft por encima de la cabeza → invoca onLeftHandRaised.
- CrossPC_KinectBridge.OnLeftHandRaised() construye "HAND_RAISED|LEFT" y lo envía 3 veces por UDP al puerto 6005 de PC A.
- Cada frame siguiente, GestureDetector invoca onLeftHandHeightChanged(valor) con la altura normalizada; CrossPC_KinectBridge.OnLeftHandHeightChanged() envía "HAND_HEIGHT|LEFT|valor" (sin redundancia) a PC A.
- En PC A, CrossPC_OculusBridge recibe HAND_RAISED|LEFT y llama a fxSelectorCh1.ActivateSelected(); luego, por cada HAND_HEIGHT|LEFT|valor recibido, llama a djManager.SetFXIntensity(1, fxSelectorCh1.ArmedEffect, valor), que mueve el parámetro continuo del efecto armado en FMOD.
- Cuando el Jugador B baja la mano, el mismo camino envía "HAND_LOWERED|LEFT" y PC A llama a fxSelectorCh1.Deactivate().

7.2 El Jugador B dice "reverb canal uno"

- VoiceEffectSelector.cs (en PC B), usando KeywordRecognizer, reconoce la frase y la asocia a (canal=1, slot=1).
- Llama a CrossPC_KinectBridge.SelectEffectByVoice(1, 1), que envía "SELECT_EFFECT|1|1" 3 veces a PC A.
- CrossPC_OculusBridge, en PC A, recibe el mensaje y llama a fxSelectorCh1.SelectEffect(1), que arma Reverb en el canal 1 (mutua exclusión con los demás efectos de ese canal, ya resuelta por el FXSelector existente).
- A partir de ahí, la altura de la mano izquierda (7.1) modula la intensidad de ese Reverb recién armado.

8. Requisitos de red y configuración

- Ambas PCs deben estar en la misma red local (mismo router/switch, mismo rango de subred); UDP de broadcast/local no cruza routers distintos sin configuración adicional.
- Firewall de Windows: la primera vez que corre cada build, Windows pregunta si permite el tráfico entrante; hay que aceptarlo. En builds sin supervisión, se debe crear una regla manual de entrada UDP para el puerto 6005 (PC A) y 6006 (PC B).
- IP fija o reservada por DHCP recomendada para ambas PCs, ya que remoteIP se configura a mano en el Inspector de cada bridge (no hay descubrimiento automático de IP en esta versión).

Campo (Inspector)	PC A — CrossPC_OculusBridge	PC B — CrossPC_KinectBridge	Notas
listenPort	6005	6006	Deben ser distintos entre sí.
remoteIP	IP de la PC B	IP de la PC A	IPv4 de la LAN, ej. 192.168.1.50
remotePort	6006	6005	Debe coincidir con el listenPort de la otra PC.
redundantSends	3	3	Veces que se repite un mensaje discreto importante.

9. Justificación de decisiones de diseño

UDP en vez de TCP/WebSockets: menor latencia para datos continuos en tiempo real (altura de mano), coherente con el canal local ya existente en el proyecto (KinectUdpReceiver), y no requiere mantener una conexión persistente ni manejar reconexión de socket.

Texto plano en vez de un formato binario: facilita depurar el tráfico con herramientas simples (Wireshark, netcat) durante el desarrollo, y mantiene el mismo estilo legible que ya usa el resto del proyecto.

Redundancia simple (2–3 envíos) en vez de ACK + retransmisión: decisión pragmática dado el tiempo disponible de un proyecto universitario; es segura porque los métodos afectados en el receptor son idempotentes, y no introduce el costo de implementar y probar un protocolo de confiabilidad propio.

Separación en dos canales UDP distintos: se decidió no tocar ni reutilizar el canal local Kinect→Unity (puerto 5005) para evitar introducir regresiones en una parte del sistema que ya funcionaba; el canal nuevo es independiente y puede probarse y depurarse por separado.

10. Limitaciones y trabajo futuro

- No hay cifrado ni autenticación en el canal: se asume una red local confiable (el mismo supuesto que ya aplicaba al canal local existente).
- No hay reconexión automática si una PC se reinicia a mitad de sesión; hay que reiniciar manualmente el componente/escena en esa PC.
- El reconocimiento de voz depende de UnityEngine.Windows.Speech, exclusivo de Windows; en otra plataforma el componente se autodesactiva mencionándolo por log, pero no hay alternativa implementada todavía.
- La lógica de negocio que decide cuándo abrir y cerrar una ventana de efectos (WINDOW_OPEN/WINDOW_CLOSE) y qué efectos ofrecer en cada canción queda del lado de PC A por integrar con el resto del flujo del juego; el protocolo ya lo soporta, pero el disparo de esos mensajes no está cableado a ningún evento de la consola todavía.
- Falta un script de prueba end-to-end entre dos PCs reales de laboratorio; hasta ahora el protocolo se diseñó y se implementó pero se recomienda validarlo primero con dos instancias en 127.0.0.1 antes de probarlo en la red física.